

Tutorial Week 2

Pre-reading/viewing: Lecture 2, document '**Software required for this module**', document '**Using AppsAnywhere guide**'

The tutorial covers the following.

1. Launching Python/IDLE on the lab PCs.
2. Conversion Functions
3. Keyboard input
4. Keyboard input with User Defined functions.
5. Practice debugging programs errors
6. Exception Handling in Python

1: Launching Python/IDLE on the lab PCs

- The lab PC's have Python 3.10.4 (includes IDLE) available via AppsAnywhere. When logged in, click on the AppsAnywhere icon:



- Then scroll down or type 'IDLE' or 'Python' in the **Search Apps** box.
- Launch version **Python 3.10.4 (NOT BARE INSTALLATION)**
- See document '**Using AppsAnywhere guide**' for further details.



Accessing Python from home.

- See 'Guide to Using AppsAnywhere on Your Personal Computer'.
- Mac Users - AppsAnywhere does not work with Macs.
- Download Python from: <https://www.python.org/downloads/>
- Any Python 3.10.x (or Python 3.9) version will be suitable.
- Or, for the first tutorial use an 'online Python 3 compiler'. E.g., <https://www.programiz.com/python-programming/online-compiler/>

2. Conversion functions

- a) The code below is designed to calculate the sum of two numbers but is giving an incorrect output. Re-write the code using the appropriate conversion function so that it outputs the correct result.

```
price1 = input('input price one? ')
price2 = input('input price two ')

total_price=price1+price2
print (f"The total price is {total_price}")
```

- b) The code below is should output the value variable as an integer.
Expected output > **The integer of the variable val1 is 27**

Rewrite this code using conversion functions so that it prints the correct output to the shell window

```
val1=27.0
print("The integer of val1 is "+val1)
```

- c) Rewrite the code below using conversion functions so the output is an integer sum of the two variables
Expected output > **The_sum is 6**

```
var1="2.7"
var2="3.3"
the_sum = var1+ var2
print("The_sum is "+the_sum)
```

3: Keyboard input

i. Try the following program:

```
name = input ('Please enter your name: \n')
print ('Hello', name)
```

- Remember, \n within quote marks forces a new line to be printed.
 - When you run the program, you should see the message "Please enter your name:" in the shell window. Type in your name, followed by the ENTER key. The program will greet you.
- ii. Extend the program to get the user's age, and print out "your age is" (with the response).
- iii. How would you print the following so that it displays as shown: test\test2\answers.txt
- iv. Type the following. Check the differences between the three print statements.

```
the_text = input('Enter some text.\n') # get some text!

#print - version 1
print('This is what you entered: ')
print(the_text)

#print - version 2
print('This is what you entered:', the_text)

#print - version 3 - Supress printing of a new line, use end=' '
print('This is what you entered:', end=' ')
print(the_text)
```

v. Lecture 2 Self-Check Questions (2-4)

Type and run programs discussed in the Lecture 2. Partial possible solutions are shown.

- **Lecture: Question 2.** Write the Python program using the pseudocode shown.

```
INPUT num_1
INPUT num_2
total <- num_1 + num_2
PRINT total
```

Partial solution shown. **Update** to convert the string input to an integer.

```
num_1 = input('Enter number one: ')
num_2 = input('Enter number two: ')
total = num_1 + num_2
print(total)
```

- **Lecture: Question 3.** Write the Python program using the pseudocode below.

```
INPUT cost_of_item
INPUT cash_paid (e.g., 10 for £10)
CALCULATE change
PRINT change
```

Partial solution shown below. **Update** to convert the string input to a float.

```
cost = input('Enter cost of item: ')
paid = input('Enter cash paid: ')
change = paid - cost
print('Change is: ', change)
```

- **Lecture: Question 4.** Program to calculate the average of three numbers (pseudocode):

```
INPUT num_1
INPUT num_2
INPUT num_3
average <- (num_1 + num_2 + num_3) / 3
PRINT average
```

Test your Python solution.

Then experiment with **round(x,y)**. Where 'x' is what you are rounding and 'y' is how many decimal places you want to round up to. E.g., `average = round (average, 2)`

4. Keyboard input continued:

Re write these *week one* practical session examples to allow a user to input any values (ask the user for input with a helpful text prompt).

i) **Use 'Input' to allow users to input any values for a,b and c**

```
a=17
b=12
c=5
result= (a+b)*c
print (f'The result is {result}')
```

ii) **Allow users to input any values for variables red, green and blue**

```
#find the average of three numbers
red=24
green=12
blue=63
greyscale_average = (red + green + blue) / 3
print (f'The greyscale value is {greyscale_average}')
```

iii) **Allow users to input any cube side length values**

```
#find the total volume of two cubes
cube_1_side=11
cube_2_side=4
total_volume=cube_1_side**3+cube_2_side**3
print (f'The total volume of both cubes is {total_volume}')
```

iv) **Allow users to input a,b and h values**

```
#find the total area of a trapezoid
a=10
b=12
h=7
Area=(a+b)*h/2
print (f'The area of the trapezoid is {Area}')
```

v) **Allow users to input any radius value**

```
#find the total volume of a sphere from the radius
sphere_radius=2
sphere_vol= 4/3 * 3.142*sphere_radius**3
print (f'The volume of the sphere is {sphere_vol}')
```

5: Practice debugging program errors

- This program adds the number of apples and oranges already available with the number bought.
- There are **four bugs in the code** for you to correct.

The debugged program should print:

```
You have:      3 apples and 0 oranges
you buy:       4 apples and 6 oranges
you now have:  7 apples and 6 oranges
```

```
# shopping program with 4 bugs
a=3          # number of apples
o=0.0        # number of oranges
print('you have:      ',a,'oranges and',o,'apples')
buy_a=4      # apples you buy
buy_o=6      # oranges you buy
print('you buy:       ',buy_a,'apples and',buy_o,'oranges')
a=a+buy_A
o=o+buy_o
PRINT('you now have:',a,'apples and',o,'oranges')
```

Tips on debugging your program errors.

Syntax error - check for missing parentheses, quotation, or commas etc.

Runtime error

- Understanding the error message will help you.
- Using extra print() statements to display the value of your program's variables can help you figure out what is happening in your program.
- Function type() will return a variable type – useful for debugging **TypeError**.
- Sometimes you need to work backwards (up) from the area indicated in the error message.

Semantic error - The program will run without error messages, but it will not do the right thing.

Additional Exercises

a. Additional exercise

Re-write the algorithms in part 4 above as user defined functions that take input values as parameter

b. Additional exercise

vi. Temperature Program (part 1) – To extend in a later tutorial. Write a program that will convert an input Centigrade temperature (c) into Fahrenheit(f).

Formula: $f = (c * 1.8) + 32$

Pseudocode:

```
INPUT centigrade
CALCULATE fahrenheit
PRINT fahrenheit
```

h) Write a program to calculate the volume of a box. Enter values for the length, height and width.

c. Additional exercise

Exception Handling in Python

a) Example: ask the user to enter a number using the input() method and cast the string input to an integer.

```
n = int(input("Please enter a number: "))
```

Type in the code, run it and enter 10.5 instead of a valid integer. If the input entered cannot be converted (cast) to an integer it will generate a ValueError.

ValueError: invalid literal for int() with base 10: '10.5'

With exception handling, we can write robust code for reading an integer from input:

```
n = input("Please enter an integer: ")
try:
    n = int(n)
    print(n)
except ValueError:
    print("Requires a valid integer!")
```

Example: The program accepts a value from the user. In the *try* clause the value in *n* is converted (cast) to an integer. If an exception occurs (the value cannot be converted to an integer) the *except* clause will be executed. The error, in this case a ValueError, has to match the name after except.

b) The following calculation would produce a **ZeroDivisionError**: $x = 45 / 0$

Create a specific try except construct that will give the message 'Cannot divide by zero' when appropriate.